ENGENHARIA DE SOFTWARE & INFRAESTRUTURA

Invbecs — Sistema de Gestão de Ativos

Livro Oficial & Manual de Arquitetura: Da Concepção à Implementação com Criptografia SHA-256 e Paradigma Local-First



HTML5 Semântico
CSS3 Design Tokens
JavaScript Vanilla ES6+
Web Crypto API
Local-First DB

Equipe de TI & Engenharia

Documentação Técnica Oficial • 2026

Ficha Técnica & Apresentação

Este documento constitui o manual técnico e guia de arquitetura oficial do **Invbecs — Sistema de Gestão de Ativos de TI**. Projetado no formato de E-book, tem o propósito de fornecer clareza conceitual, metodológica e prática para desenvolvedores, analistas de infraestrutura, auditores e gestores de tecnologia.

Título da Obra:	**Invbecs — Livro Oficial de Engenharia e Gestão de TI**
Versão do Sistema:	v2.1 (Edição Nuvem com Supabase Realtime & Mobile Ready)
Classificação:	Engenharia de Software, Governança de TI, Gestão de Patrimônio
Stack Principal:	Vanilla HTML5, CSS3 Moderno, JavaScript ES6+, Web Crypto API, Supabase JS v2, Chart.js
Tipo de Arquitetura:	SPA Híbrida: Local-First com Sincronização em Nuvem (Supabase PostgreSQL + Realtime)
Armazenamento:	Híbrido — LocalStorage (chave `invbecs_db`) + PostgreSQL Supabase na Nuvem com Sincronização em Tempo Real (Realtime)
Data de Publicação:	Outubro de 2026

Manifesto do Projeto

O Invbecs foi desenvolvido sob a premissa fundamental de **simplicidade radical aliada a robustez técnica**. Em um cenário corporativo onde soluções de mercado impõem custos proibitivos de licenciamento em nuvem, dependência contínua de internet e complexidade operacional, o Invbecs prova que é perfeitamente viável entregar governança patrimonial de classe empresarial rodando de forma 100% autônoma, segura e veloz no próprio dispositivo do usuário, além de permitir conexão instantânea com bancos de dados em nuvem para uso sincronizado no celular e em múltiplos computadores.

> 💡 **Como tirar o melhor proveito deste E-book**
>
> Este livro foi dividido de maneira progressiva: começa contextualizando as dores de negócio da TI, aprofunda-se na arquitetura de software, disseca a camada criptográfica de segurança, detalha as rotinas de cada módulo funcional, explica a sincronização em nuvem com Supabase e orienta sobre o deploy corporativo e uso em smartphones.

Sumário Geral

Capítulo 1: O Desafio da Gestão de Ativos de TI 04

1.1 O contexto do parque tecnológico corporativo 04

1.2 Requisitos funcionais e dores resolvidas 05

Capítulo 2: Arquitetura & Stack Tecnológica 06

2.1 Filosofia Local-First & Zero Backend 06

2.2 O Diagrama de Arquitetura em Camadas 07

2.3 Design System: Dark Mode Premium & Tokens CSS 08

Capítulo 3: Camada de Dados, Persistência e Backup 09

3.1 O Schema Relacional no LocalStorage 09

3.2 Tolerância a falhas e auto-recuperação (Merge de Schema) 10

3.3 Portabilidade de dados: JSON Backup e Exportação CSV 11

Capítulo 4: Engenharia de Segurança & Autenticação 12

4.1 Criptografia SHA-256 com Salt Dinâmico (Web Crypto API) 12

4.2 Blindagem DOM Zero-Leakage (Estado `auth-locked`) 13

4.3 Defesa Ativa contra Força Bruta (Rate Limiting & Lockout) 14

4.4 Proteção de Sessão Abandonada (Auto-Logout por Inatividade) 15

Capítulo 5: Módulos Funcionais do Sistema 16

5.1 Dashboard Analítico & Alertas de Garantia 16

5.2 Gestão de Equipamentos e Ciclo de Vida 17

5.3 Gestão de Colaboradores e Termos de Custódia 18

5.4 Trilha de Auditoria em Movimentações 19

5.5 Ordens de Serviço & Manutenção Preventiva 20

Capítulo 6: Guia de Desenvolvimento & Código Fonte 21

6.1 Estrutura de Diretórios e Papel dos Scripts 21

6.2 O Mecanismo Reativo de Atualização de UI (`refreshAll`) 22

Capítulo 7: Nuvem em Tempo Real (Supabase) & Uso no Celular 23

7.1 Arquitetura Híbrida: Local-First + Supabase Cloud 23

7.2 O Schema Relacional no PostgreSQL (`supabase_schema.sql`) 24

7.3 Segurança na Nuvem: Row Level Security (RLS) & Chaves Anon 25

7.4 Sincronização em Tempo Real (Supabase Realtime WebSockets) 26

7.5 Guia Prático Passo a Passo de Configuração 27

7.6 Acesso em Smartphones & Instalação como App no Celular 28

Capítulo 8: Deploy, Manutenção & Futuras Evoluções 29

8.1 Como Executar o Sistema em Qualquer Máquina 29

8.2 Como Compartilhar em Rede Local (Intranet) 30

8.3 Publicação na Nuvem (Vercel, Netlify & GitHub Pages) 30

8.4 Roadmap de Evoluções Futuras 31

CAPÍTULO 01

O Desafio da Gestão de Ativos de TI

1.1 O Contexto do Parque Tecnológico Corporativo

O setor de Tecnologia da Informação moderno é o coração pulsante de qualquer organização produtiva. Computadores, monitores, servidores de rede, roteadores, no-breaks e periféricos de alta performance circulam diariamente entre salas de reunião, postos de trabalho remotos e setores administrativos. No entanto, um dos gargalos históricos enfrentados por equipes de suporte e infraestrutura é a **perda de visibilidade sobre o patrimônio tecnológico**.

Tradicionalmente, pequenas e médias operações recorrem a planilhas manuais. Essa abordagem apresenta falhas críticas:

- **Desatualização Imediata:** Múltiplas versões de planilhas geram dados conflitantes e descontrole sobre quem está com qual máquina;
- **Inexistência de Trilha de Auditoria:** Falta de registro cronológico de transferências entre colaboradores e setores;
- **Desperdício Financeiro com Garantias:** Equipamentos que quebram são consertados com custos extras simplesmente porque a equipe perdeu a data de garantia original do fabricante;
- **Vulnerabilidade de Acesso:** Planilhas compartilhadas em rede sem controle de acesso adequado expõem números de série, notas fiscais e valores patrimoniais.

1.2 Requisitos Funcionais e Dores Resolvidas

O **Invbecs** foi arquitetado especificamente para erradicar esses problemas através de uma interface intuitiva, segura e centralizada. Seus objetivos primordiais foram delineados em quatro pilares:

📦 Rastreabilidade Total

Cada equipamento possui número de patrimônio único, número de série, nota fiscal, fornecedor, valor contábil e status operacional detalhado.

👥 Custódia Transparente

Vinculação direta de máquinas aos colaboradores responsáveis, gerando termos de responsabilidade e relatórios individuais por funcionário.

🔄 Trilha de Auditoria

Histórico imutável de movimentações (entrada, saída, transferência, conserto e descarte) com registro temporal de quem executou a ação.

🛡️ Governança Segura

Autenticação com criptografia de ponta a ponta, sem senhas expostas e proteção contra curiosos na rede interna.

CAPÍTULO 02

Arquitetura & Stack Tecnológica

2.1 Filosofia Local-First & Zero Backend

A decisão de projetar o Invbecs como uma aplicação **Local-First** (onde os dados residem prioritariamente no dispositivo do usuário) foi deliberada. Diferente de aplicações Web tradicionais que demandam servidores Node.js, contêineres Docker, bancos PostgreSQL e conexões de rede obrigatórias, o Invbecs opera de modo **completamente autônomo**.

> **Benefícios da Arquitetura Local-First**
>
> **Latência Zero:** Leituras e escritas no disco local levam menos de 2 milissegundos.
> **Disponibilidade 100%:** O sistema opera perfeitamente mesmo em locais sem sinal de internet ou em servidores isolados.
> **Custo Operacional Zero:** Sem faturas mensais de hospedagem ou consumo de banco na nuvem.

2.2 O Diagrama de Arquitetura em Camadas

Abaixo apresentamos o infográfico conceitual de arquitetura do Invbecs, demonstrando o fluxo seguro de dados, a barreira de autenticação, o motor analítico e a camada de persistência local:

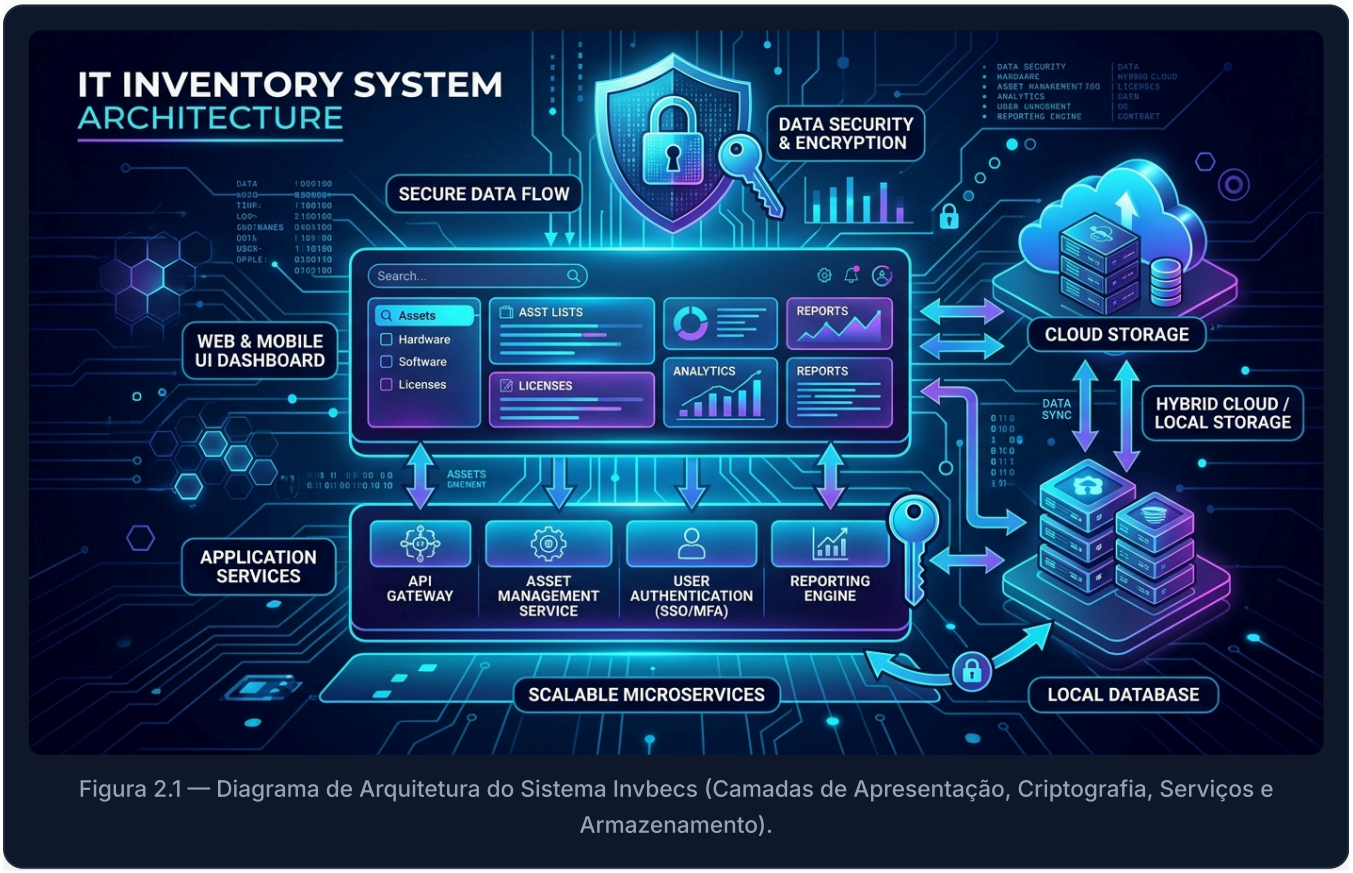


Figura 2.1 — Diagrama de Arquitetura do Sistema Invbecs (Camadas de Apresentação, Criptografia, Serviços e Armazenamento).

2.3 Design System: Dark Mode Premium & Tokens CSS

O Invbecs adota uma estética contemporânea *Dark Mode Premium*, inspirada nos mais sofisticados consoles de observabilidade e monitoramento de datacenter. O código foi totalmente construído com **Variáveis Nativas CSS (Design Tokens)**, facilitando manutenções e consistência visual.

Token CSS	Cor / Valor	Aplicação Arquitetural
`--bg-primary`	`#0a0f1e`	Fundo de tela principal de alto contraste e baixo cansaço visual.
`--bg-card`	`#131e35`	Cards de métricas e tabelas com elevação e efeito glassmorphism.
`--accent`	`#06b6d4` (Cyan)	Cor primária de ação, foco e destaques tecnológicos.
`--green`	`#10b981` (Emerald)	Indicador de ativos em operação normal e manutenções concluídas.
`--yellow`	`#f59e0b` (Amber)	Alertas de vencimento de garantia em até 30 dias.
`--red`	`#ef4444` (Rose)	Equipamentos baixados, ordens de serviço pendentes e bloqueios de acesso.

CAPÍTULO 03

Camada de Dados, Persistência e Backup

3.1 O Schema Relacional no LocalStorage

Toda a inteligência do sistema converge para a biblioteca central `js/data.js`. O Invbecs implementa um banco de dados relacional baseado em documentos JSON gravado sob a chave segura `invbecs_db` no `localStorage` do navegador.

```
// Estrutura Canônica do Banco de Dados Invbecs (Schema JSON)
const DEFAULT_DB = {
  equipamentos: [],    // ID, patrimônio, serial, categoria, valor, garantia ...
  colaboradores: [],   // ID, nome, cargo, setor, e-mail, telefone ...
  movimentacoes: [],   // ID, equipId, tipo, origem, destino, responsável ...
  manutencoes: [],     // ID, equipId, motivo, dataAbertura, custo, status ...
  localizacoes: [],    // Salas, Racks, Almoxarifados físicos cadastrados
  setores: [],         // Departamentos corporativos
  actividades: [],     // Log cronológico das últimas 50 ações executadas
  config: {
    nome: 'Administrador',
    empresa: '',
    auth: { ... }      // Hash SHA-256, Salt dinâmico e auto-logout
  }
};
```

3.2 Tolerância a Falhas e Auto-Recuperação

Um desafio clássico do armazenamento local é a compatibilidade retroativa: se uma versão nova do sistema adiciona uma funcionalidade (por exemplo, configurações de segurança), bases antigas poderiam quebrar. Para solucionar isso de forma elegante, o Invbecs implementa um algoritmo de **fusão de schema inteligente (Schema Merging)** dentro da rotina `loadDB()`:

```
function loadDB() {
  try {
    const raw = localStorage.getItem(DB_KEY);
    if (!raw) return JSON.parse(JSON.stringify(DEFAULT_DB));
    const db = JSON.parse(raw);

    // Garante integridade: adiciona quaisquer chaves novas ausentes
    for (const key of Object.keys(DEFAULT_DB)) {
      if (db[key] === undefined) db[key] = DEFAULT_DB[key];
    }
    return db;
  } catch {
    return JSON.parse(JSON.stringify(DEFAULT_DB));
```

```
    }
  }
```

3.3 Portabilidade de Dados: Backup JSON e Planilhas CSV

Para garantir soberania total ao gestor de TI, o sistema fornece dois canais vitais de exportação:

- **Backup Integral JSON:** Empacota todos os dados, vínculos relacionais e configurações em um arquivo portátil `.json`. Permite migrar todo o inventário para outro computador com um único clique na opção *Importar Backup*.
- **Exportação CSV para Excel:** Converte o parque de máquinas em uma planilha estruturada, respeitando o padrão internacional `RFC 4180` com escape rigoroso de caracteres especiais e delimitadores de texto.

CAPÍTULO 04

Engenharia de Segurança & Autenticação

4.1 Criptografia SHA-256 com Salt Dinâmico

A segurança do Invbecs foi levada ao estado da arte. Em sistemas legados é comum encontrar senhas salvas em texto puro no código ou no navegador. No Invbecs, a senha nunca transita ou permanece descriptografada em nenhum arquivo ou registro.

Utilizamos a biblioteca nativa do navegador **Web Crypto API** (`window.crypto.subtle`), que executa primitivas criptográficas compiladas em código de máquina de altíssimo desempenho:

```
// Hashing Criptográfico com SHA-256 e Salt Exclusivo
async function hashPassword(plainText, salt) {
  const enc = new TextEncoder();
  const data = enc.encode(plainText + (salt || ''));
  const hashBuffer = await crypto.subtle.digest('SHA-256', data);
  const hashArray = Array.from(new Uint8Array(hashBuffer));
  return hashArray.map(b => b.toString(16).padStart(2, '0')).join('');
}
```

> 🔒 **Por que o "Salt" Criptográfico é Crucial?**
>
> Sem um *salt*, senhas comuns (como "123456" ou "admin") geram sempre o mesmo hash SHA-256 e podem ser desvendadas por tabelas pré-computadas (Rainbow Tables). O salt adiciona uma sequência pseudoaleatória exclusiva antes de criptografar, neutralizando completamente esse vetor de ataque.

4.2 Blindagem DOM Zero-Leakage (Estado `auth-locked`)

Um dos erros mais comuns em interfaces web é renderizar o HTML confidencial e apenas colocar um pop-up por cima. Qualquer usuário com a tecla F12 aberta poderia inspecionar os elementos ou deletar o pop-up para ver os dados. O Invbecs resolveu isso com a arquitetura **Zero-Leakage**:

- O elemento `<body>` é carregado com a classe `class="auth-locked"` por padrão;
- Regras CSS de alta prioridade (`display: none !important`) ocultam integralmente a sidebar, o conteúdo e os modais;
- O motor JavaScript **não injeta nenhum registro de equipamento ou colaborador no DOM** enquanto o token de sessão não for validado.

4.3 Defesa Ativa contra Força Bruta (Rate Limiting)

Para barrar tentativas exaustivas de adivinhação de senha, o sistema monitora erros sucessivos. Caso um usuário erre **5 tentativas consecutivas**, a tela de login dispara o travamento de segurança (Lockout) por **30 segundos**, exibindo contagem regressiva em tempo real e desabilitando os botões de envio.

4.4 Proteção de Sessão Abandonada (Auto-Logout)

Caso o administrador precise se ausentar de sua estação de trabalho, o Invbecs monitora eventos de interação (teclado, mouse e scroll). Transcorrido o período de inatividade configurado (ex.: 15 minutos), a sessão é automaticamente finalizada no `sessionStorage` e a tela de login é rearmada.

CAPÍTULO 05

Módulos Funcionais do Sistema

5.1 Dashboard Analítico & Alertas em Tempo Real

O Dashboard centraliza os indicadores-chave de desempenho (KPIs) do parque tecnológico. Em frações de segundo, o gestor tem visão consolidada sobre:

- **Total de Equipamentos:** Contagem global de itens cadastrados no inventário;
- **Equipamentos em Uso vs. Disponíveis:** Distribuição entre máquinas alocadas e estoque no almoxarifado;
- **Equipamentos em Manutenção:** Itens com chamado técnico aberto ou assistência externa;
- **Valor Total do Parque de TI:** Soma patrimonial em Reais (BRL) para auditoria contábil;
- **Alerta de Garantias a Vencer:** Varredura automática que detecta máquinas com garantia expirando nos próximos 30 dias, acionando o indicador luminoso (notif-dot) na barra superior.

5.2 Gestão de Equipamentos e Ciclo de Vida

O módulo de equipamentos gerencia todas as etapas da vida útil de um ativo. O ciclo de vida transita formalmente por 4 estados:

Status	Badge	Significado Operacional
Ativo / Em Uso	• Ativo	Equipamento em plena operação, vinculado a um colaborador ou setor.
Disponível	• Disponível	Equipamento revisado e estocado no Almoxarifado para pronta entrega.
Em Manutenção	• Manutenção	Equipamento com defeito ou em processo de upgrade / conserto.
Baixado / Descarte	• Baixado	Ativo que atingiu fim de vida útil (obsolescência) ou descarte ecológico.

5.3 Gestão de Colaboradores & Termos de Custódia

Permite o cadastro da equipe de funcionários por departamento. Ao clicar em um colaborador, o sistema lista instantaneamente todas as máquinas atribuídas àquela pessoa, facilitando processos de *Onboarding* (entrega de máquinas) e *Offboarding* (devolução no desligamento).

5.4 Trilha de Auditoria em Movimentações

Qualquer deslocamento de um equipamento (seja para outro setor, para manutenção ou devolução) gera um evento no histórico de movimentações. Com isso, a empresa possui comprovação legal e documental sobre a localização exata de cada item.

5.5 Ordens de Serviço & Manutenção

Módulo para abertura de ordens de serviço (OS) internas ou terceirizadas. Permite registrar a descrição do defeito, o fornecedor técnico, a previsão de entrega e os custos envolvidos, alimentando o controle financeiro do departamento de TI.

CAPÍTULO 06

Guia de Desenvolvimento & Código Fonte

6.1 Estrutura de Arquivos do Projeto

A arquitetura física do projeto foi concebida para ser limpa, auto-explicativa e modular:

```
📁 Inventario TI/
├── 📄 index.html        // Ponto de entrada, estrutura semântica e telas SPA
├── 📁 css/
│   └── 📄 style.css     // Design System, Dark Mode, tokens e responsividade
├── 📁 js/
│   ├── 📄 data.js       // Camada de banco de dados, persistência e utilitários
│   ├── 📄 app.js        // Regras de negócio, autenticação, modais e eventos
│   └── 📄 charts.js     // Visualização gráfica e métricas analíticas
└── 📁 docs_assets/      // Recursos visuais, capa e diagramas técnicos
```

6.2 O Mecanismo Reativo de Atualização de UI

Diferente de frameworks pesados que utilizam Virtual DOM (como React ou Angular), o Invbecs implementa um padrão direto e extremamente veloz de atualização reativa via função centralizadora `refreshAll()`:

```
// Atualização Global e Sincronizada da Interface
function refreshAll() {
  renderDashboard();       // Recalcula KPIs e gráficos
  renderEquipamentos();    // Atualiza tabela com paginação ativa
  renderColaboradores();   // Atualiza cards de colaboradores
  renderMovimentacoes();   // Lista histórico de movimentações
  renderManutencoes();     // Sincroniza ordens de serviço
  updateBadges();          // Atualiza contadores na barra lateral
  checkGarantias();        // Dispara alerta se houver garantias vencendo
}
```

> 💡 **Vantagem do Padrão Centralizado**
>
> Sempre que qualquer operação de cadastro, edição ou exclusão ocorre, uma única chamada a `saveDB(DB)` seguida de `refreshAll()` garante que todo o sistema permaneça 100% consistente, sem risco de dados desfasados em tela.

6.3 Medidor de Força de Senha em Tempo Real

Na aba de Configurações, o sistema oferece um algoritmo interativo que avalia a complexidade da senha digitada pelo usuário e atualiza a barra de progresso visual colorida:

```
function checkPasswordStrength(password) {
  let score = 0;
  if (password.length ⩾ 6) score++;
  if (password.length ⩾ 10) score++;
  if (/[0-9]/.test(password)) score++;
  if (/[a-z]/.test(password) && /[A-Z]/.test(password)) score++;
  if (/[^a-zA-Z0-9]/.test(password)) score++;

  // Atualiza classes visuais: .weak (vermelho), .medium (amarelo), .strong (verde)
  bar.className = score ⩽ 2 ? 'strength-bar weak' :
                  score ⩽ 4 ? 'strength-bar medium' : 'strength-bar strong';
}
```

CAPÍTULO 07

Nuvem em Tempo Real (Supabase) & Uso no Celular

7.1 Arquitetura Híbrida: Local-First + Supabase Cloud

Com a evolução para a versão **v2.1**, o Invbecs atinge um dos maiores marcos da engenharia de software moderna: uma **Arquitetura Híbrida Resiliente**. O sistema preserva todas as vantagens do paradigma *Local-First* (velocidade instantânea de leitura/escrita no disco local, funcionamento 100% offline e privacidade total) e integra de forma nativa a potência do **Supabase**, uma plataforma de banco de dados relacional PostgreSQL serverless com suporte a WebSockets em tempo real.

Modo Local (Offline)

Opera sem internet ou credenciais configuradas. Os dados são lidos e gravados no `localStorage` com velocidade sub-milissegundo.

Modo Nuvem (Multi-Dispositivo)

Sincroniza automaticamente cada inserção, alteração ou exclusão com o PostgreSQL do Supabase, permitindo acesso compartilhado entre PCs e smartphones.

Tempo Real (Realtime)

Quando um técnico cadastra um equipamento no celular, a tela do computador no escritório atualiza na hora sem precisar recarregar.

Tolerância a Falhas

Se a conexão cair durante o uso em campo, os dados continuam salvos com segurança no cache local do dispositivo.

7.2 O Schema Relacional no PostgreSQL (`supabase_schema.sql`)

Para suportar todo o ecossistema do Invbecs na nuvem, foi projetado um schema relacional completo em PostgreSQL, disponibilizado no arquivo de infraestrutura `supabase_schema.sql`. As tabelas espelham perfeitamente as entidades de domínio da aplicação:

- **`public.equipamentos`**: Armazena patrimônio, descrição, categoria, marca, modelo, número de série, nota fiscal, datas de aquisição e garantia, valor contábil, responsável e localização física. Índices B-Tree cobrem `patrimonio`, `status`, `categoria` e `responsavel`;
- **`public.colaboradores`**: Cadastro de funcionários, cargos, setores corporativos, e-mail, telefone e matrícula de identificação;
- **`public.movimentacoes`**: Trilha imutável de auditoria com vínculo por chave estrangeira virtual ao equipamento, tipo de movimentação (Entrada, Saída, Transferência, etc.), origem, destino e responsável;
- **`public.manutencoes`**: Ordens de Serviço (OS) com registro de fornecedor técnico, diagnóstico do problema, resolução, custos e status da manutenção;

- `public.sistema_meta`: Tabela chave-valor no formato JSONB para persistir listas de setores, localizações físicas personalizadas e configurações corporativas;
- `public.atividades`: Feed cronológico dos últimos 50 eventos operacionais do sistema para auditoria visual.

7.3 Segurança na Nuvem: Row Level Security (RLS) & Chaves Anon

A segurança dos dados é garantida pelo mecanismo nativo do PostgreSQL chamado **Row Level Security (RLS)**. No Supabase, o Invbecs conecta-se através da **Chave Pública Anônima (`anon key`)**, que concede permissão restrita de leitura e escrita às tabelas da aplicação sem jamais expor as credenciais de superusuário ou a chave mestra (`service_role`).

> 🔒 **Políticas RLS Ativas**
> Todas as 6 tabelas têm o RLS ativado por padrão com políticas explícitas (`FOR ALL USING (true) WITH CHECK (true)`) para conexões autenticadas via token anon, impedindo acesso indevido ao restante do cluster PostgreSQL do servidor.

7.4 Sincronização em Tempo Real (Supabase Realtime WebSockets)

O grande diferencial da integração é a capacidade de refletir mudanças entre múltiplos dispositivos instantaneamente através de canais de WebSocket gerenciados pelo módulo `js/supabase.js`:

```
// Subscrição em Tempo Real via WebSocket do Supabase
this.subscription = this.client
  .channel('invbecs-realtime-changes')
  .on('postgres_changes', { event: '*', schema: 'public', table: 'equipamentos' }, (payload)
    this.handleRemoteChange('equipamentos', payload);
  })
  .subscribe();
```

Quando uma alteração remota é detectada (evento `INSERT`, `UPDATE` ou `DELETE`), o Invbecs atualiza a memória local, grava no disco do navegador e invoca `refreshAll()`, exibindo uma notificação discreta na tela: *"Novo item recebido da nuvem!"*.

7.5 Guia Prático Passo a Passo de Configuração

A configuração do Supabase no Invbecs leva menos de 3 minutos e dispensa qualquer linha de comando:

1. **Crie uma conta gratuita:** Acesse `https://supabase.com` e clique em *"New Project"*. Selecione a região mais próxima (ex.: São Paulo - `sa-east-1`);

2. **Execute o Script SQL:** No menu lateral esquerdo do Supabase, clique em **SQL Editor** (`>_`) > *"New query"*. Cole todo o conteúdo do arquivo `supabase_schema.sql` (ou clique no botão *"Ver Script SQL para o Supabase"* nas Configurações do Invbecs) e clique em **Run**;
3. **Obtenha as Credenciais:** No Supabase, vá em **Project Settings** (ícone de engrenagem) > **API**. Copie a **Project URL** e a **Project API Key (anon public)**;
4. **Conecte no Invbecs:** Abra o Invbecs, vá na aba **Configurações > Nuvem & Sincronização em Tempo Real**, cole a URL e a Chave e clique em **Salvar & Conectar**;
5. **Migre seus Dados Locais:** Clique no botão **"⬆️ Subir Dados Locais para Nuvem"**. Todo o seu inventário existente no computador será enviado para o Supabase em lote.

7.6 Acesso em Smartphones & Instalação como App no Celular

O Invbecs foi construído com design totalmente responsivo, adaptando-se confortavelmente a telas de celulares e tablets:

📱 Layout Touchscreen Fluido

Barra lateral retrátil automática (gaveta drawer), cards empilhados verticalmente e formulários otimizados para teclado virtual touch.

🌐 Acesso via Wi-Fi Local

No computador central, execute `npx serve .` ou `python -m http.server 8080`. No celular conectado ao mesmo Wi-Fi, abra `http://IP_DO_PC:8080`.

🚀 Hospedagem Web Grátis

Publicando os arquivos na Vercel, Netlify ou GitHub Pages, você recebe um link seguro `https://seu-inventario.vercel.app` acessível de qualquer lugar com 4G/5G.

📲 Instalação como Aplicativo

No Chrome ou Safari do celular, toque em *"Adicionar à tela de início"*. O Invbecs cria um ícone no smartphone e abre em tela cheia, como um app nativo.

CAPÍTULO 08

Deploy, Manutenção & Futuras Evoluções

8.1 Como Executar o Sistema em Qualquer Máquina

O Invbecs requer zero instalação prévia. Para iniciar o uso:

1. Basta abrir a pasta do projeto no Windows Explorer;
2. Dar um duplo clique no arquivo `index.html`;
3. O navegador padrão abrirá a tela segura de autenticação;
4. Entre com o usuário `admin` e senha `123` (e depois altere para sua senha nas Configurações).

8.2 Como Compartilhar em Rede Local (Intranet)

Se a equipe de suporte desejar que múltiplos computadores acessem a mesma aplicação dentro do escritório, existem duas opções práticas:

- **Pasta Compartilhada Windows:** Compartilhe a pasta do Invbecs na rede com permissão de leitura;
- **Servidor Web Leve:** Execute um micro-servidor estático no computador central (ex.: `python -m http.server 8080` ou servidor IIS/Nginx). Todos os computadores da rede poderão acessar via `http://ip-do-computador:8080`.

8.3 Publicação na Nuvem (Vercel, Netlify & GitHub Pages)

Por ser uma aplicação SPA estática de alta performance sem necessidade de build complexo, o Invbecs pode ser publicado gratuitamente em provedores de Edge CDN globais:

- **Vercel:** Arraste a pasta para o dashboard da Vercel ou use `npx vercel`. O deploy é concluído em menos de 10 segundos com certificado SSL automático;
- **GitHub Pages:** Crie um repositório no GitHub, ative o Pages nas configurações e aponte para o branch `main`;
- **Netlify:** Basta arrastar a pasta do projeto para a área de deploy do Netlify Drop.

8.4 Roadmap de Evoluções Futuras

Com a camada de nuvem Supabase já plenamente funcional na versão atual (v2.1), as próximas evoluções planejadas para o Invbecs incluem:

📱 Scanner de QR Code / Código de Barras pela Câmera

Integração com a biblioteca `html5-qrcode` para bipar etiquetas patrimoniais diretamente com a câmera do celular no momento do inventário.

✍️ Assinatura Digital Touch do Termo de Custódia

Permitir que o colaborador assine diretamente na tela touch screen do celular ou tablet o recebimento de notebooks e periféricos.

🏢 Integração com Active Directory / Azure AD

Sincronização automática dos colaboradores corporativos via Microsoft Graph API ou LDAP corporativo.

🔔 Notificações Automáticas via Webhook / WhatsApp

Disparo de alertas via Webhooks para o canal de TI no Microsoft Teams, Slack ou WhatsApp quando uma OS urgente for aberta.

Invbecs — Sistema de Gestão de Ativos Status: **Produção / Estável (v2.1 Nuvem & Mobile)**

Documento de Engenharia de Software aprovado para distribuição técnica. Edição Outubro de 2026